

REPORTE LAB S05

Paso 1 · Clona el repo

En primer lugar, realizó la clonación del repositorio mexi-banco en la versión v05.1

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~
$ git clone --branch v05.1 https://github.com/OscarRuiz21/mexi-banco.git
Cloning into 'mexi-banco'...
remote: Enumerating objects: 123, done.
remote: Counting objects: 100% (123/123), done.
remote: Compressing objects: 100% (97/97), done.
remote: Total 123 (delta 23), reused 107 (delta 12), pack-reused 0 (from 0)
Receiving objects: 100% (123/123), 46.71 KiB | 82.00 KiB/s, done.
Resolving deltas: 100% (23/23), done.
warning: refs/tags/v05.1 cd46b8b91a09070f8262c3b8acfb17caf2405cda is not a commit!
Note: switching to '88203161aa31a371823715684a548d154e1516f8'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false
```

Paso 2 · Levántalo con Compose

Se construyeron y levantaron los contenedores utilizando docker compose up --build -d. Tras corregir un conflicto de saltos de línea (CRLF a LF) propio de Windows en el archivo mvnw, el proceso de compilación terminó con éxito.

Al ejecutar docker compose ps, se comprobó que tanto el contenedor de la aplicación (app) como el de la base de datos (db) alcanzaron el estado healthy. Esto demuestra la utilidad de la instrucción depends_on con service_healthy, garantizando que la aplicación solo arrancara hasta que Postgres estuviera completamente listo para aceptar conexiones.

```
#20 resolving provenance for metadata file
#20 DONE 0.0s
[+] up 5/5
✓ Image mexi-banco-app Built 110.8s
✓ Network mexibanco Created 0.1s
✓ Volume mexi-banco_mexibanco_datos Created 0.0s
✓ Container mexi-banco-db-1 Healthy 6.1s
✓ Container mexi-banco-app-1 Started 6.1s
```



```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco ((v05.1))
$ docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS                PORTS
mexi-banco-app-1    mexi-banco-app      "java -jar app.jar"    app        19 seconds ago  Up 12 seconds (healthy)  0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-db-1     postgres:17-alpine  "docker-entrypoint.s..." db         19 seconds ago  Up 18 seconds (healthy)  5432/tcp
```

Paso 3 · Prueba los endpoints

3.1 Salud de la aplicación

Se consultó el endpoint `/actuador/health` para validar que el servicio estaba arriba y listo para recibir peticiones, devolviendo un estado "UP".

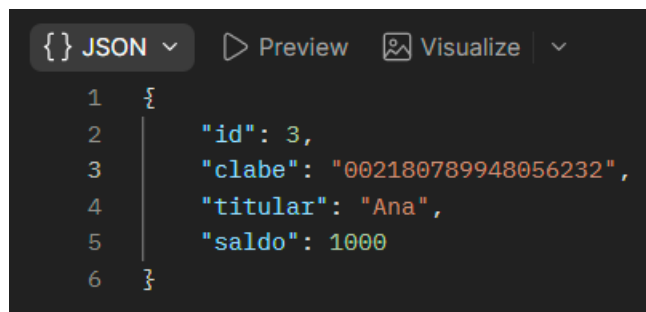


```
{
  "status": "UP",
  "groups": [
    "liveness",
    "readiness"
  ]
}
```

3.2 Apertura de cuentas

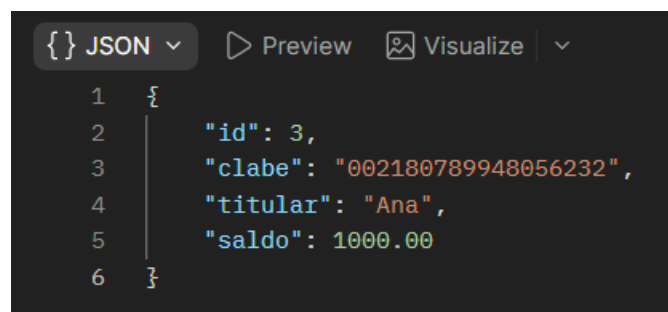
Se crearon exitosamente dos cuentas. La cuenta origen a nombre de Ana con un saldo inicial de \$1000, y la cuenta destino a nombre de Beto con un saldo inicial de \$500. Se comprobó la creación consultando la cuenta origen por su CLABE.

Abrir cuenta de origen



```
{
  "id": 3,
  "clabe": "002180789948056232",
  "titular": "Ana",
  "saldo": 1000
}
```

Consultar cuenta origen:



```
{
  "id": 3,
  "clabe": "002180789948056232",
  "titular": "Ana",
  "saldo": 1000.00
}
```

Abrir cuenta destino:

```
{ } JSON ▾ ▶ Preview 🖼 Visualize ▾  
1 {  
2   "id": 4,  
3   "clabe": "012180789948056232",  
4   "titular": "Beto",  
5   "saldo": 500  
6 }
```

3.3 Transferencia y validación de consistencia

Se ejecutó el endpoint de transferencias para enviar \$200 desde la cuenta origen a la cuenta destino. Al consultar nuevamente los saldos, se verificó que el sistema mantiene la consistencia de los datos: a Ana se le descontaron \$200 (saldo final \$800) y a Beto se le sumaron \$200 (saldo final \$700). La transacción quedó registrada correctamente con su respectivo ID y estampa de tiempo.

Transferir 200 de origen a destino:

```
{ } JSON ▾ ▶ Preview 🖼 Visualize ▾  
1 {  
2   "id": 2,  
3   "claveOrigen": "002180789948056232",  
4   "claveDestino": "012180789948056232",  
5   "monto": 200,  
6   "fecha": "2026-09-20T23:50:24.299744039Z"  
7 }
```

Consultar origen (después de transferencia):

```
{ } JSON ▾ ▶ Preview 🖼 Visualize ▾  
1 {  
2   "id": 3,  
3   "clabe": "002180789948056232",  
4   "titular": "Ana",  
5   "saldo": 800.00  
6 }
```

Consultar cuenta destino (después de transferencia):

```
{ } JSON v Preview Visualize v
1 {
2   "id": 4,
3   "clabe": "012180789948056232",
4   "titular": "Beto",
5   "saldo": 700.00
6 }
```

Consultar transferencia:

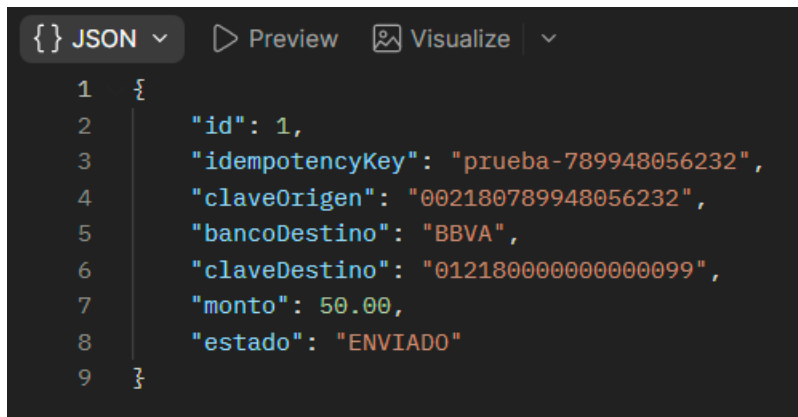
```
{ } JSON v Preview Visualize v
1 {
2   "id": 2,
3   "claveOrigen": "002180789948056232",
4   "claveDestino": "012180789948056232",
5   "monto": 200.00,
6   "fecha": "2026-09-20T23:50:24.299744Z"
7 }
```

3.4 Idempotencia

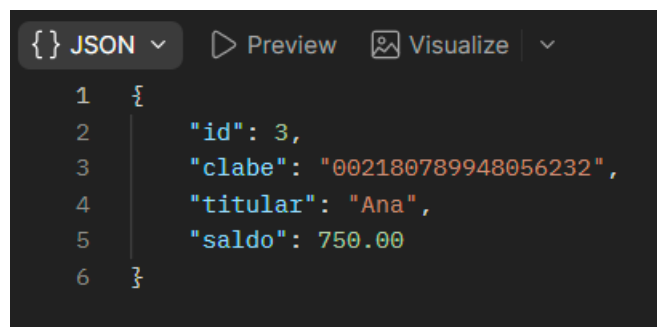
Posteriormente pasamos a probar la parte de idempotencia mandando el mismo SPEI dos veces con la misma Idempotency-Key verificando que se cobre una sola vez.

Primer envío

```
{ } JSON v Preview Visualize v
1 {
2   "id": 1,
3   "idempotencyKey": "prueba-789948056232",
4   "claveOrigen": "002180789948056232",
5   "bancoDestino": "BBVA",
6   "claveDestino": "012180000000000099",
7   "monto": 50,
8   "estado": "ENVIADO"
9 }
```

Segundo envío (reintento)

```
{
  "id": 1,
  "idempotencyKey": "prueba-789948056232",
  "claveOrigen": "002180789948056232",
  "bancoDestino": "BBVA",
  "claveDestino": "012180000000000099",
  "monto": 50.00,
  "estado": "ENVIADO"
}
```

Consultamos cuenta origen

```
{
  "id": 3,
  "clabe": "002180789948056232",
  "titular": "Ana",
  "saldo": 750.00
}
```

En este punto observamos que a pesar de haber enviado dos veces la transferencia, solo se le descontó la cantidad de 50 a Ana, se respetó la Idempotency-Key y el cobro se realizó una sola vez.

3.5 El monolito por dentro (Estado de cuenta y avisos)

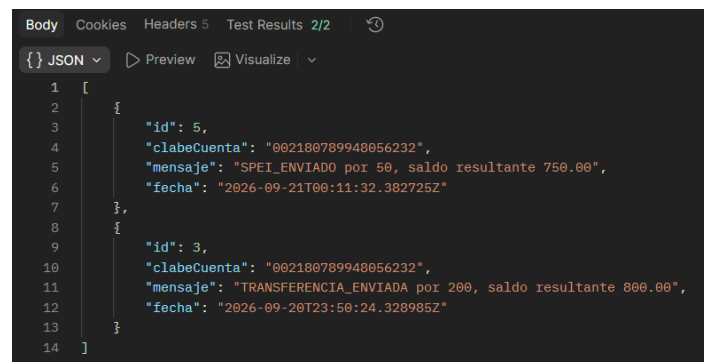
Para comprobar la comunicación interna entre las distintas capas lógicas del monolito, se consultaron los módulos de movimientos y notificaciones.

- **Estado de cuenta del origen:** Al ejecutar el método GET en `/movimientos?clabe={{origen}}`, el sistema devolvió el historial (ledger) de la cuenta origen, reflejando correctamente el depósito inicial y el cargo por la transferencia realizada en el paso anterior.



```
1  [
2    {
3      "id": 9,
4      "clabeCuenta": "002180789948056232",
5      "tipo": "SPEI_ENVIADO",
6      "monto": -50.00,
7      "saldoResultante": 750.00,
8      "fecha": "2026-09-21T00:11:31.726560Z",
9      "referencia": "BBVA · 01218000000000099"
10   },
11   {
12     "id": 7,
13     "clabeCuenta": "002180789948056232",
14     "tipo": "TRANSFERENCIA_ENVIADA",
15     "monto": -200.00,
16     "saldoResultante": 800.00,
17     "fecha": "2026-09-20T23:50:24.296732Z",
18     "referencia": "a 012180789948056232"
19   },
20   {
21     "id": 5,
22     "clabeCuenta": "002180789948056232",
23     "tipo": "DEPOSITO",
24     "monto": 1000.00,
25     "saldoResultante": 1000.00,
26     "fecha": "2026-09-20T23:47:37.613313Z",
27     "referencia": "saldo inicial"
28   }
29 ]
```

- **Avisos del origen:** Al ejecutar el método GET en /notificaciones?clabe={{origen}}, se verificó que el "stub" de notificaciones registró exitosamente los eventos (como el alta de la cuenta y el aviso de transferencia), demostrando que las operaciones en el módulo de Cuentas disparan acciones en el módulo de Notificaciones dentro del mismo proceso.



```
1  [
2    {
3      "id": 5,
4      "clabeCuenta": "002180789948056232",
5      "mensaje": "SPEI_ENVIADO por 50, saldo resultante 750.00",
6      "fecha": "2026-09-21T00:11:32.382725Z"
7    },
8    {
9      "id": 3,
10     "clabeCuenta": "002180789948056232",
11     "mensaje": "TRANSFERENCIA_ENVIADA por 200, saldo resultante 800.00",
12     "fecha": "2026-09-20T23:50:24.328985Z"
13   }
14 ]
```

3.6 Cuando algo sale mal (Manejo de errores de negocio)

Se realizaron pruebas de estrés enviando peticiones malformadas o inválidas según las reglas de negocio. El objetivo fue comprobar que la aplicación no colapsa (código HTTP 500 Internal Server Error), sino que intercepta la falla y responde con el código HTTP adecuado según el estándar REST.

- **Cuenta inexistente (404 Not Found):** Al intentar consultar una CLABE que no existe en la base de datos (99999999999999999999), el sistema respondió correctamente con un 404, indicando que el recurso no fue encontrado.

```
{ } JSON ▾ ▶ Preview 🐞 Debug with AI ▾  
1 {  
2   "type": "about:blank",  
3   "title": "Not Found",  
4   "status": 404,  
5   "detail": "No existe la CLABE 99999999999999999999",  
6   "instance": "/cuentas/99999999999999999999"  
7 }
```

- **CLABE repetida (409 Conflict):** Al intentar abrir una cuenta nueva utilizando la misma CLABE de la cuenta origen, el sistema arrojó un código 409, protegiendo la integridad de la base de datos ante duplicados.

```
{ } JSON ▾ ▶ Preview 🐞 Debug with AI ▾  
1 {  
2   "type": "about:blank",  
3   "title": "Conflict",  
4   "status": 409,  
5   "detail": "Ya existe una cuenta con la CLABE 002180789948056232",  
6   "instance": "/cuentas"  
7 }
```

- **Cuerpo incompleto (400 Bad Request):** Al enviar el cuerpo de la petición (Payload) sin el atributo obligatorio de la CLABE, la aplicación rechazó la solicitud por sintaxis inválida con un 400.

```
{ } JSON ▾ ▶ Preview 🐞 Debug with AI ▾  
1 {  
2   "timestamp": "2026-09-21T00:26:38.241+00:00",  
3   "status": 400,  
4   "error": "Bad Request",  
5   "path": "/cuentas"  
6 }
```

- **Saldo insuficiente (422 Unprocessable Entity):** Se intentó transferir un monto exagerado (999999) que superaba el saldo de la cuenta origen. El sistema validó la regla de negocio y bloqueó la operación devolviendo un error 422.

```
{ } JSON ▾ ▶ Preview 🐞 Debug with AI ▾  
1 {  
2   "type": "about:blank",  
3   "title": "Unprocessable Entity",  
4   "status": 422,  
5   "detail": "Saldo insuficiente en la CLABE 002180789948056232",  
6   "instance": "/transferencias"  
7 }
```

- **Transferencia a la misma cuenta (422 Unprocessable Entity):** Se intentó enviar dinero colocando la misma CLABE en origen y destino, lo cual fue rechazado exitosamente con un código 422 por violar las reglas lógicas del negocio.

```
{ } JSON Preview Debug with AI
1 {
2   "type": "about:blank",
3   "title": "Unprocessable Entity",
4   "status": 422,
5   "detail": "La cuenta origen y la destino son la misma",
6   "instance": "/transferencias"
7 }
```

- **SPEI sin Idempotency-Key (400 Bad Request):** Se intentó mandar un pago externo (SPEI) omitiendo la cabecera/parámetro de Idempotencia. Como se vio en la teoría, esta llave es obligatoria para evitar cobros dobles en operaciones con *timeouts*, por lo que el sistema la rechazó con un 400.

```
{ } JSON Preview Debug with AI
1 {
2   "timestamp": "2026-09-21T00:27:55.901+00:00",
3   "status": 400,
4   "error": "Bad Request",
5   "path": "/spei"
6 }
```

3.7 Comprobación de atomicidad y consistencia:

Finalmente, se volvió a consultar el saldo de la cuenta origen mediante un método GET. Se comprobó que el saldo se mantuvo intacto a pesar de todas las transacciones fallidas anteriores. Esto demuestra que las operaciones del sistema son **atómicas**: si la validación falla, toda la transacción se deshace (rollback) y no se generan efectos secundarios (cobros fantasma).

```
{ } JSON Preview Visualize
1 {
2   "id": 3,
3   "clabe": "002180789948056232",
4   "titular": "Ana",
5   "saldo": 750.00
6 }
```

Paso 4 · Bájalos con y sin volumen

Se realizaron pruebas sobre la persistencia de datos administrando el ciclo de vida de los contenedores y sus volúmenes asociados.

Prueba 1 (docker compose down): Se apagaron los contenedores y se volvieron a levantar. Al consultar la cuenta origen en Postman, los datos y el saldo seguían intactos. Esto ocurre porque el comando down por defecto solo destruye los contenedores, pero preserva el volumen de Docker donde Postgres guarda físicamente la información, como ya habíamos visto en clase.

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco ((v05.1))
$ docker compose down
[+] down 3/3
✓ Container mexi-banco-app-1 Removed 1.8s
✓ Container mexi-banco-db-1 Removed 0.4s
✓ Network mexibanco Removed 0.2s

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco ((v05.1))
$ docker compose up -d
[+] up 3/3
✓ Network mexibanco Created 0.1s
✓ Container mexi-banco-db-1 Healthy 6.2s
✓ Container mexi-banco-app-1 Started 6.3s
```

```
{ } JSON ▾ ▶ Preview 🖼 Visualize ▾
1 {
2   "id": 3,
3   "clabe": "002180789948056232",
4   "titular": "Ana",
5   "saldo": 750.00
6 }
```

Prueba 2 (docker compose down -v): Se apagaron los contenedores agregando la bandera -v (volumes) y se volvieron a levantar. Al consultar la misma cuenta, el sistema respondió con un error 404 Not Found. Esto ocurre porque la bandera -v destruye explícitamente el volumen. Al levantar el entorno nuevamente, Postgres inicia una base de datos completamente en blanco.

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco ((v05.1))
$ docker compose down -v
[+] down 4/4
✓ Container mexi-banco-app-1 Removed 2.0s
✓ Container mexi-banco-db-1 Removed 0.4s
✓ Volume mexi-banco_mexibanco_datos Removed 0.1s
✓ Network mexibanco Removed 0.3s

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco ((v05.1))
$ docker compose up -d
[+] up 4/4
✓ Network mexibanco Created 0.0s
✓ Volume mexi-banco_mexibanco_datos Created 0.0s
✓ Container mexi-banco-db-1 Healthy 6.0s
✓ Container mexi-banco-app-1 Started 6.1s
```

```
{ } JSON ▾ ▶ Preview 🐞 Debug with AI ▾
1 {
2   "type": "about:blank",
3   "title": "Not Found",
4   "status": 404,
5   "detail": "No existe la CLABE 002180789948056232",
6   "instance": "/cuentas/002180789948056232"
7 }
```